



PR2: Uninformed Search Algorithms – BFS and DFS

Practical: Implement BFS and DFS for route-finding or maze navigation and compare path length, nodes explored and execution time.

1. Aim and Objectives

Aim: To implement Breadth First Search (BFS) and Depth First Search (DFS) algorithms for solving a route-finding problem and compare their performance based on **path length**, **nodes explored** and **execution time**.



Understand graph-based search problems.



Implement BFS using a queue.



Implement DFS using a stack.



Find a route from source to destination.

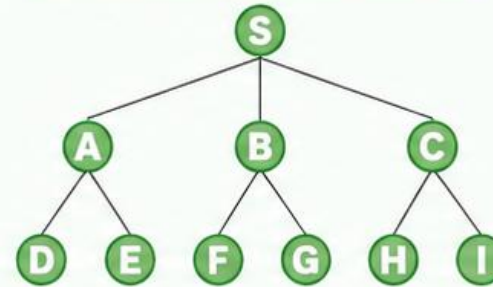


Calculate path length and nodes explored.



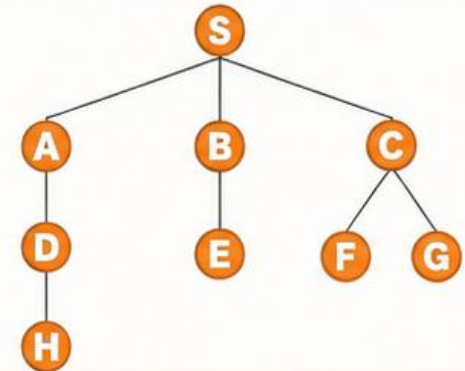
Measure and compare execution time.

BFS (Breadth First Search)



- Explores level by level
- Uses Queue (FIFO)
- Finds shortest path in unweighted graphs

DFS (Depth First Search)



- Explores as deep as possible
- Uses Stack (LIFO)
- May not find shortest path

2. Problem Statement



Represent a **map as a graph** where **locations are nodes** and **roads are edges**.

Given a **source** and **destination**, implement **BFS** and **DFS** to find a route and compare both algorithms using **path length**, **nodes explored** and **execution time**.

3. Graph and Route-Finding Concepts



Graph: A collection of vertices (nodes) connected by edges. In route finding, locations are nodes and roads are edges.



Route finding: Finding a sequence of connected nodes from a source to a goal.

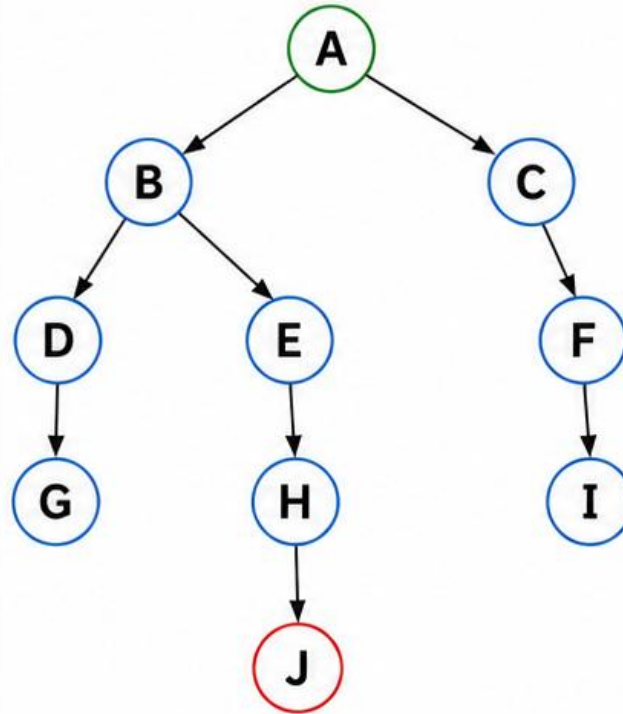


Start = A



Goal = J

Graph Representation



Graph as Adjacency List (Dictionary)

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': ['G'],  
    'E': ['H'],  
    'F': ['I'],  
    'G': [],  
    'H': ['J'],  
    'I': [],  
    'J': []  
}
```

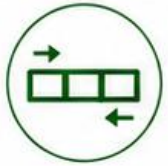
4. Uninformed Search



Uninformed or **blind search** does not use **heuristic information** about how close a state is to the goal. It relies on the **graph structure**, **available actions** and **goal test**.

Examples: BFS, DFS and Uniform Cost Search. This practical focuses on BFS and DFS.

5. Detailed BFS Theory, Algorithm and Steps

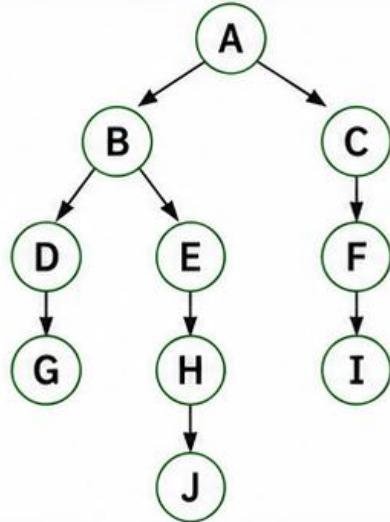


BFS explores a graph level by level. It uses a **queue** following **FIFO** (First In, First Out). In an unweighted graph, BFS guarantees a **shortest path** in terms of number of edges.

BFS idea:

Level 0: A
Level 1: B, C
Level 2: D, E, F
Level 3: G, H, I

S Start = A
G Goal = J



BFS(Graph, Start, Goal)

- 6 Create an empty queue.
- 7 Add **Start**.
- 8 Create visited set.
- 9 While queue is not empty:
 - a Remove first path.
 - b Get current node.
 - c Skip if already visited.
 - d Mark visited and count it.
 - e If current is **Goal**, return path.
 - f Add paths through unvisited neighbours.
- 10 If queue is empty, no path exists.

Graph and Route-Finding Concepts



Graph: A collection of vertices (nodes) connected by edges. In route finding, locations are nodes and roads are edges.



Route finding: Finding a sequence of connected nodes from a source to a goal.

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': ['G'],  
    'E': ['H'],  
    'F': ['I'],  
    'G': [],  
    'H': ['J'],  
    'I': [],  
    'J': []  
}
```



BFS Python Implementation



Purpose

Finds a route from start to goal in a graph using Breadth First Search (BFS).
Also counts the number of nodes explored.



Parameters

- **graph** : Dictionary representing the graph
- **start** : Starting node
- **goal** : Goal node



Returns

- **(path, nodes_explored)** :
path → List of nodes from start to goal (shortest path)
nodes_explored → Total number of nodes explored during BFS.



Key Idea

- Uses a queue (FIFO) to explore level by level.
- Visits nodes in the order they are discovered.
- In an unweighted graph, BFS guarantees the **shortest path**.

```
from collections import deque
```

```
def bfs(graph, start, goal):
```

```
    queue = deque([[start]])
```

```
    visited = set()
```

```
    nodes_explored = 0
```

```
    while queue:
```

```
        path = queue.popleft()
```

```
        current = path[-1]
```

```
        if current in visited:
```

```
            continue
```

```
        visited.add(current)
```

```
        nodes_explored += 1
```

```
        if current == goal:
```

```
            return path, nodes_explored
```

```
        for neighbour in graph[current]:
```

```
            if neighbour not in visited:
```

```
                queue.append(path + [neighbour])
```

```
    return None, nodes_explored
```

```
# BFS function
```

```
# Create queue and add start path
```

```
# Set to keep track of visited nodes
```

```
# Counter for explored nodes
```

```
# Loop until queue is empty
```

```
# Remove first path (FIFO)
```

```
# Get the last node in current path
```

```
# Skip if already visited
```

```
# Mark current as visited
```

```
# Increment explored nodes count
```

```
# Goal found
```

```
# Return path and count
```

```
# Explore neighbours
```

```
# If neighbour not visited
```

```
# Add new path to queue
```

```
# No path found
```



Detailed DFS Theory, Algorithm and Python Implementation

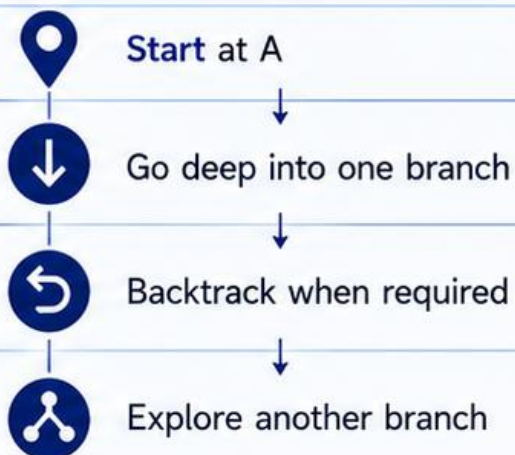


DFS explores **one branch as deeply as possible** before backtracking.

Iterative DFS uses a **stack** following **LIFO (Last In, First Out)**.

DFS **does not** guarantee the shortest path.

DFS Idea



Key Points

- Uses a stack (LIFO).
- Goes as deep as possible in one direction.
- Backtracks when a dead end is reached.
- **Does not** guarantee shortest path.

DFS(Graph, Start, Goal) – Algorithm

- 1 Create an empty stack.
- 2 Push **Start**.
- 3 Create visited set.
- 4 While stack is not empty:
 - a Pop a path.
 - b Get current node.
 - c Skip if already visited.
 - d Mark visited and count it.
 - e If current is **Goal**, return path.
 - f Push paths through unvisited neighbours.
- 5 If stack is empty, no path exists.

Python Implementation (DFS)

```
def dfs(graph, start, goal):
    stack = [[start]]
    visited = set()
    nodes_explored = 0

    while stack:
        path = stack.pop()
        current = path[-1]

        if current in visited:
            continue

        visited.add(current)
        nodes_explored += 1

        if current == goal:
            return path, nodes_explored

        for neighbour in graph[current]:
            if neighbour not in visited:
                stack.append(path + [neighbour])

    return None, nodes_explored
```



Complete Practical Program: BFS and DFS for Route-Finding

1. Graph (Map Representation)

```
from collections import deque
import time

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['G'],
    'E': ['H'],
    'F': ['I'],
    'G': [],
    'H': ['J'],
    'I': [],
    'J': []
}
```

Graph Structure

Locations are nodes and roads are edges.

Start = 'A'

Goal = 'J'

A simple map used for route-finding using BFS and DFS.

2. BFS Function

```
def bfs(graph, start, goal):
    queue = deque([[start]])
    visited = set()
    nodes_explored = 0

    while queue:
        path = queue.popleft()
        current = path[-1]
        if current in visited:
            continue

        visited.add(current)
        nodes_explored += 1

        if current == goal:
            return path, nodes_explored

        for neighbour in graph[current]:
            if neighbour not in visited:
                queue.append(path + [neighbour])

    return None, nodes_explored
```

3. DFS Function

```
def dfs(graph, start, goal):
    stack = [[start]]
    visited = set()
    nodes_explored = 0

    while stack:
        path = stack.pop()
        current = path[-1]
        if current in visited:
            continue

        visited.add(current)
        nodes_explored += 1

        if current == goal:
            return path, nodes_explored

        for neighbour in graph[current]:
            if neighbour not in visited:
                stack.append(path + [neighbour])

    return None, nodes_explored
```

4. Main Program – Run and Compare BFS & DFS

```
start, goal = 'A', 'J'

t = time.perf_counter()
bfs_path, bfs_nodes = bfs(graph, start, goal)
bfs_time = time.perf_counter() - t

t = time.perf_counter()
dfs_path, dfs_nodes = dfs(graph, start, goal)
dfs_time = time.perf_counter() - t
```

```
print("----- BFS -----")
print("Path:", " -> ".join(bfs_path) if bfs_path else "No path")
print("Path Length:", len(bfs_path)-1 if bfs_path else "N/A")
print("Nodes Explored:", bfs_nodes)
print("Execution Time:", bfs_time, "seconds")

print("\n----- DFS -----")
print("Path:", " -> ".join(dfs_path) if dfs_path else "No path")
print("Path Length:", len(dfs_path)-1 if dfs_path else "N/A")
print("Nodes Explored:", dfs_nodes)
print("Execution Time:", dfs_time, "seconds")
```

5. Sample Output (Example Run)

----- BFS -----

Path: A -> B -> E -> H -> J

Path Length: 4

Nodes Explored: 10

Execution Time: 0.000042 seconds

----- DFS -----

Path: A -> B -> D -> G

(no path from G to J)

A -> B -> E -> H -> J

Path Length: 4

Nodes Explored: 8

Execution Time: 0.000031 seconds

Notes

- BFS guarantees the shortest path in an unweighted graph.
- DFS may find a longer path and does not guarantee optimality.
- Results (nodes explored & time) can vary slightly on different runs.

Program Explanation and Key Concepts

8. Code Explanation

- `deque` is used to implement the BFS queue efficiently.
- `queue.popleft()` gives FIFO behaviour for BFS.
- `stack.pop()` gives LIFO behaviour for DFS.
- `visited` stores nodes already explored and avoids repeated processing.
- `nodes_explored` counts processed nodes.
- `path[-1]` gives the current node.
- `path + [neighbour]` creates a new route.
- `time.perf_counter()` measures elapsed execution time.



9. Path Length Calculation

For a path stored as nodes, path length in terms of moves/edges is one less than the number of nodes.



Path: A -> B -> E -> H -> J

Nodes = 5

Edges/moves = 4

```
path_length = len(path) - 1
```

10. Nodes Explored

A node is counted when it is removed from the queue/stack, confirmed as unvisited, marked visited and processed.

```
visited.add(current)
nodes_explored += 1
```



The number depends on graph structure, source, goal, neighbour order and search strategy.

11. Execution-Time Measurement

```
start_time = time.perf_counter()
path, nodes = bfs(graph, start, goal)
execution_time = time.perf_counter() - start_time
```



`time.perf_counter()` returns a high-resolution timer and is used to calculate the elapsed time for BFS and DFS execution.

12 BFS vs DFS Comparison

Parameter	BFS	DFS
 Strategy	Level-by-level	Depth-first
 Data structure	Queue	Stack / recursion
 Order	FIFO	LIFO
 Shortest path in unweighted graph	Guaranteed	Not guaranteed
 Path length	Optimal in unweighted graph	May be longer
 Nodes explored	Depends on graph and goal	Depends on graph and goal
 Memory	Usually higher	Usually lower



Summary: Use BFS when you need the shortest path in an unweighted graph.
Use DFS when memory is limited or when exploring deep paths is more important.

13

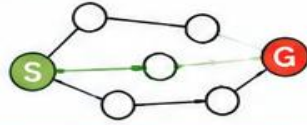
Applications of BFS and DFS



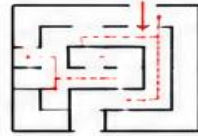
BFS and **DFS** are fundamental graph traversal techniques used in many real-world applications.

BFS Applications

- 1 Shortest path in unweighted graphs



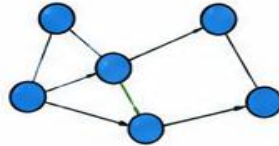
- 2 Route and maze navigation



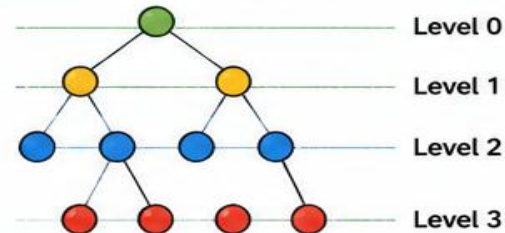
- 3 Network broadcasting



- 4 Minimum number of connections

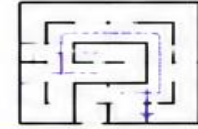


- 5 Finding nodes at a particular distance

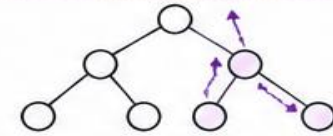


DFS Applications

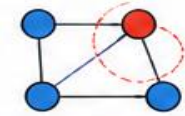
- 1 Maze exploration



- 2 Backtracking



- 3 Cycle detection



- 4 Connected components



- 5 Topological sorting



- 6 Puzzle and state-space exploration



- 7 Checking path existence

